1/1/2026

Gnome's Well

A structured, step-by-step instructional framework for developing a complete 2D game using the Unity engine, intended to facilitate the acquisition of fundamental game development concepts, including user interface design, animation systems, physics implementation, particle systems, and audio integration.

Submitted by:

Ancheta, Arem

Lardizabal, Timothy

Tan, Bryan

Submitted to:

Jayson Nacorda

Table of Contents

Getting Started Building the Game

Knowing how to navigate Unity's interface is one thing. Creating an entire game with it is another.By the end of this part, you'll have Gnome's Well That Ends Well, a side scrolling action game. We'll start by creating the project in Unity, and do a little bit of setup. We'll also import some of the assets that will be needed in the early stages; as we progress, more will be imported**.**

Part 1: Creating the Project and Importing Assets

Step 1: Create a New Unity Project

1. Open Unity Hub.
2. Click New Project.
3. Select 2D (Core) as the template.
4. Set the Project Name to GnomesWell.
5. Choose a location on your computer where the project will be saved.
6. Make sure no asset packages are checked.
7. Click Create Project.

Step 2: Download the Game Assets

1. Open your browser and go to:
   https://www.secretlab.com.au/books/unity
2. Download the art, sound, and resource files for the project.
3. Extract (unzip) the downloaded file into a folder you can easily find.

Do not drag everything into Unity yet. We will import assets gradually.

Step 3: Save the Scene

1. In Unity, go to File → Save As.
2. Name the scene Main.
3. Make sure it is saved inside the Assets folder.
4. Click Save.

Tip: From now on, pressing Ctrl + S (or Cmd + S on Mac) will quickly save your work.

Step 4: Create Project Folders (Organization Step)

Keeping your project organized will save you time later.

1. In the Project window, right-click on the Assets folder.
2. Select Create → Folder.

3. Create the following folders:

- Scripts – for all C# code files
- Sounds – for music and sound effects
- Sprites – for all images used in the game
- Gnome – for gnome prefabs and related objects
- Level – for level backgrounds, walls, and traps
- App Resources – for app icons and splash screens

When finished, your Assets folder should look neat and organized.

Step 5: Import the Prototype Gnome Sprites

1. Locate the Prototype Gnome folder inside the downloaded assets.
2. Drag the Prototype Gnome folder into: Assets → Sprites in Unity.

These are temporary sprites that help us build the character before replacing it with polished art.

Part 2: Creating the Gnome Character

The gnome is made of multiple moving parts, so we need a parent object to hold everything together.

Step 1: Create the Parent Gnome Object

1. Go to GameObject → Create Empty.
2. Rename the object to Prototype Gnome.
3. Select the object, then in the Inspector:
    - Set the Tag to Player.
4. In the Transform component:
    - Set Position X, Y, Z = 0.
    - If not zero, click the ⚙ icon → Reset.

Why Player tag?

This allows the game to detect when the gnome touches traps, treasure, or exits.

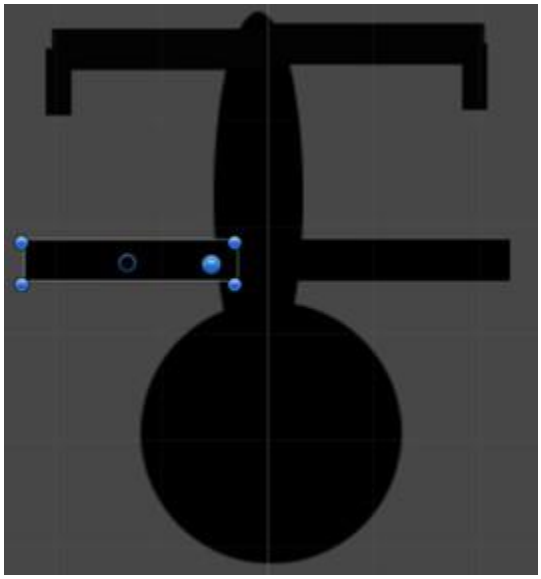
Step 2: Add the Gnome Sprites to the Scene

1. Open Sprites → Prototype Gnome.

2. Drag the following sprites one at a time into the Scene:

- Prototype Arm Holding
- Prototype Arm Loose
- Prototype Body
- Prototype Head
- Prototype Leg Dangle
- Prototype Leg Rope

+ *Do NOT drag all sprites at once.* Unity may create an animation by mistake.



Step 3: Make the Sprites Children of the Gnome

1. In the Hierarchy, select all gnome sprites.
2. Drag them onto Prototype Gnome.

Your hierarchy should now show all sprites nested under the parent object.

Step 4: Position the Body Parts

1. Switch to the Move Tool (press T or click the arrow icon).
2. Carefully move the arms, legs, and head so they connect naturally to the body.
3. For each sprite:
    - Set Tag = Player
    - Set Z Position = 0

This ensures correct layering and collision behavior.

Step 5: Add Rigidbody 2D Components

Rigidbody components allow objects to be affected by physics.

1. Select all body part sprites (not the parent).
2. In the Inspector, click Add Component.
3. Search for Rigidbody 2D and add it.

- *Important:* Do NOT add a regular Rigidbody (3D). Always use Rigidbody 2D.

Step 6: Add Colliders (Physical Shapes)

Colliders define how objects physically interact.

- Arms & Legs → Add BoxCollider2D
- Head → Add CircleCollider2D (leave default size)
- Body → Add CircleCollider2D
    - Reduce the radius to better fit the body sprite

Part 3: Connecting the Body Parts (Joints)

We will use HingeJoint2D so body parts can rotate naturally.

Step 1: Add Hinge Joints

1. Select all sprites except the body.
2. Click Add Component → Physics 2D → Hinge Joint 2D.

Step 2: Connect Joints to the Body

1. In the HingeJoint2D settings:
2. Drag Prototype Body into Connected Rigidbody.

This links all parts to the body.

Step 3: Add Rotation Limits

This prevents unnatural movement.

- Arms & Head
    - Enable Use Limits
    - Lower Angle: -15
    - Upper Angle: 15
- Legs
    - Enable Use Limits
    - Lower Angle: -45
    - Upper Angle: 0

Step 4: Fix Joint Pivot Points

By default, joints rotate from the center (which looks wrong).

1. Select a body part with a hinge joint.
2. In the Scene view:
    - Blue Dot = Connected Anchor
    - Blue Circle = Anchor
3. Move both anchors to the correct pivot:
    - Arms → shoulders

- Legs → hips
- Head → base of neck

Tip: If the anchor is stuck in the center, adjust its values in the Inspector first.

Part 4: Adding the Rope Spring Joint

This allows the gnome to hang from a rope.

Step 1: Select the Rope Leg

1. Select Prototype Leg Rope (top-right leg).

Step 2: Add Spring Joint 2D

1. Click Add Component → Spring Joint 2D.
2. Move the Anchor (blue circle) near the foot.
3. Do NOT move the Connected Anchor.

Step 3: Configure the Spring

- Disable Auto Configure Distance
- Distance: 0.01

- Frequency: 5

This creates a flexible, bouncy rope effect.

Step 4: Test the Game

1. Click Play .
2. The gnome should now dangle naturally.

Part 5: Resize the Gnome

1. Select Prototype Gnome (parent).
2. In the Transform component:
    - Set Scale X = 0.5
    - Set Scale Y = 0.5

This scales the gnome to the correct size for the level.

Rope System Tutorial

The rope works by chaining together multiple small objects (called rope segments) using physics joints. Each segment is connected to the next, forming a flexible rope. The top of the rope stays fixed, and the bottom connects to the gnome's rope leg.

Part 1: Creating the Rope Segment Prefab

This prefab will act as the **building block** for the rope.

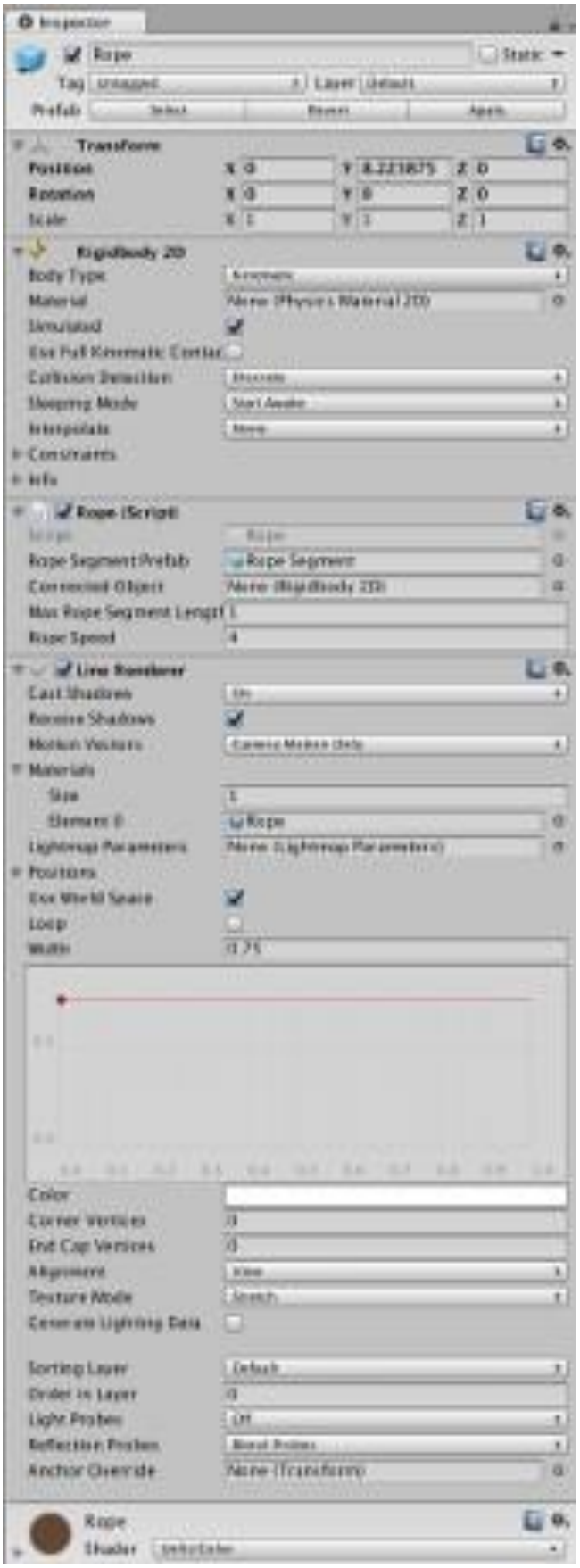
Step 1: Create the Rope Segment Object

1. Go to GameObject → Create Empty.
2. Rename the object to Rope Segment.

This object will not be visible—it exists only for physics.

Step 2: Add a Rigidbody2D

1. Select Rope Segment.
2. Click Add Component.
3. Add Rigidbody 2D.
4. Set the following value:
    - Mass = 0.5


Rope
Static
Tag
Layer
Prefab
Select
Revert
Apply
Transform
Position
Rotation
Scale
Rigidbody 2D
Body Type
Material
Simulated
Use Full Kinematic Contac
Collision Detection
Sleeping Mode
Interpolate
Constraints
Info
Rope (Script)
Rope Segment Prefab
Rope Segment
Connected Object
Max Rope Segment Lengt
Rope Speed
Line Renderer
Cast Shadows
Receive Shadows
Motion Vectors
Materials
Size
Element 0
Rope
Lightmap Parameters
Positions
Use World Space
Loop
Width
Color
Corner Vertices
End Cap Vertices
Alignment
Texture Mode
Generate Lighting Data
Sorting Layer
Default
Order in Layer
Light Probes
Off
Reflection Probes
Anchor Override
None (Transform)
Rope
Shader

Step 3: Add a Spring Joint 2D

1. With Rope Segment still selected, click Add Component.
2. Add Spring Joint 2D.
3. Set these values:
    - Damping Ratio = 1
    - Frequency = 30

These values make the rope stable but still flexible. You can tweak them later if you want different behavior.

Step 4: Create the Rope Segment Prefab

1. Open the Gnome folder in the Project window.
2. Drag Rope Segment from the Hierarchy into the Gnome folder.

You have now created a Rope Segment prefab.

Step 5: Delete the Scene Object

1. Delete the Rope Segment object from the Hierarchy.

We don't need it in the scene anymore—the script will create copies automatically.

Part 2: Creating the Rope Controller Object

This object controls the entire rope.

Step 1: Create the Rope Object

1. Go to GameObject → Create Empty.
2. Rename it Rope.

Step 2: Change the Rope Icon (Visibility Tip)

The rope object has no sprite, so we give it an icon to make it easy to see.

1. Select the Rope object.
2. Click the cube icon at the top-left of the Inspector.
3. Choose the red rounded rectangle (pill shape).

This helps you find the Rope object in the Scene view.

Step 3: Add a Rigidbody2D (Fixed in Place)

1. Select Rope.
2. Click Add Component → Rigidbody 2D.
3. Change Body Type to Kinematic.

Why Kinematic?

This prevents the rope's anchor point from falling due to gravity.

Step 4: Add a Line Renderer

The Line Renderer draws the visible rope.

1. Click Add Component → Line Renderer.
2. Set:
    - Width = 0.075
3. Leave all other settings at default.

Part 3: Creating the Rope Script

Now we add the script that builds, grows, shrinks, and draws the rope.

Step 1: Add the Rope Script

1. Select the Rope object.
2. Click Add Component.
3. Type Rope.
4. Select New Script.
5. Ensure:
    - Language: C Sharp
    - Name: Rope (capital R)
6. Click Create and Add.

Unity will create Rope.cs and attach it automatically.

Step 2: Move the Script to the Scripts Folder

1. In the Project window, locate Rope.cs.
2. Drag it into the Scripts folder.

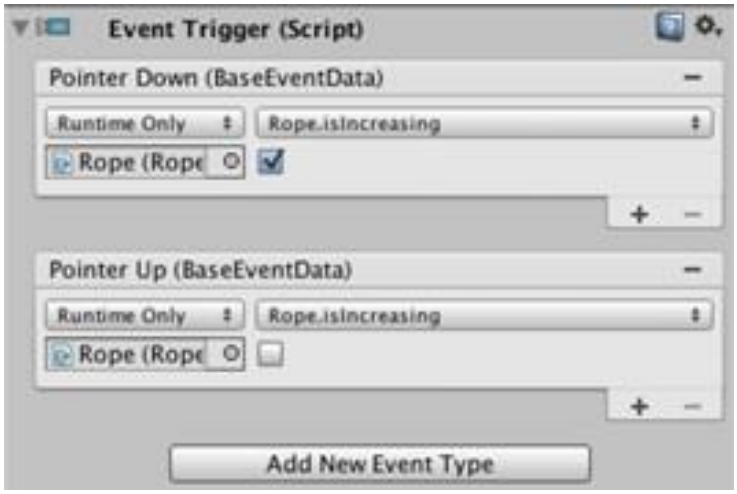
This keeps your project organized.

Part 4: Understanding the Rope Code

You don't need to memorize the code—just understand what it does.

What Happens When the Game Starts

- The Start() method runs.
- It calls ResetLength().
- The Line Renderer is linked to the Rope object.



ResetLength()

This method:

- Deletes all existing rope segments
- Clears the rope segment list
- Resets growth/shrink states
- Creates a brand-new rope

It is also used when the gnome dies.

CreateRopeSegment()

This method:

- Creates a new Rope Segment
- Connects it to the previous segment
- Attaches the top segment to the Rope object
- Attaches the first segment to the gnome's Rope Leg

RemoveRopeSegment()

This method:

- Deletes the top rope segment
- Reconnects the next segment to the Rope object
- Never removes the last segment, so the rope never disappears completely

Update()

Runs every frame:

- If **isIncreasing = true**
  - The rope length increases
  - A new segment is added when needed
- If **isDecreasing = true**
  - The rope shortens
  - Segments are removed when distance is near zero
- The Line Renderer updates its points to match the rope segments

Part 5: Configuring the Rope in the Scene

Step 1: Assign Required References

1. Select the Rope object.
2. In the Inspector:
   - Drag Rope Segment prefab into Rope Segment Prefab
   - Drag the gnome's Prototype Leg Rope into Connected Object

Leave all other values as default.

Step 2: Test the Rope

1. Click Play
2. The gnome should now hang from the rope.
3. You should see a thin line connecting the gnome to the anchor point.

Part 6: Creating the Rope Material

The rope is visible, but we want it to look better.

Step 1: Create the Material

1. In the Project window, right-click in Assets.
2. Choose Create → Material.
3. Name it Rope.

Step 2: Set the Rope Color

1. Select the Rope material.
2. In the Inspector:
    - Shader → Unlit → Color
3. Click the color box.
4. Choose a dark brown color.

Step 3: Assign the Material to the Line Renderer

1. Select the Rope object.
2. In the Line Renderer → Materials section:
    - Drag the Rope material into Element 0

Step 4: Final Test

1. Click Play ▶ again.
2. The rope should now appear brown and realistic.

Rope system complete!

You now have a physics-based rope connected to the gnome, fully controlled by code.

Preparing for Gameplay

Now that the gnome and the rope are working, we can start making the game interactive. In this section, we will:

1. Set up input testing using Unity Remote
2. Add tilt controls so the gnome swings left and right
3. Create an InputManager using the Singleton pattern
4. Add a Swinging script that applies force to the gnome

Everything here is explained step by step and in simple terms.

Part 1: Understanding Player Input

Mobile games use touch and motion sensors instead of keyboards.

In this game:

- The player tilts the phone
- The accelerometer detects left–right movement
- The gnome swings based on that tilt

To test this without constantly building the app, we use Unity Remote.

Part 2: Setting Up Unity Remote

What Is Unity Remote?

Unity Remote is a mobile app that lets your phone act as an input device for the Unity Editor.

When the game is playing in Unity:

- Your phone shows the game screen
- Touch and motion data are sent back to the Editor

This lets you test quickly without building the app every time.

Limitations of Unity Remote (Important to Know)

- The video stream is compressed, so quality is lower
- There may be input delay (latency)
- Performance is not exactly the same as a real build
- The phone must stay connected by USB

Despite this, Unity Remote is excellent for development and testing.

How to Use Unity Remote

1. Download Unity Remote from your phone's app store
2. Connect your phone to your computer using a USB cable
3. Open the Unity Remote app on your phone

4. In Unity, click Play ▶

You should now see the game appear on your phone.

If Nothing Appears on Your Phone

1. Go to Edit → Project Settings → Editor
2. In the Device dropdown:
    - Select your connected phone Then press Play ▶ again.

Part 3: Adding Tilt Control

Tilt control uses two scripts:

- InputManager – reads accelerometer data
- Swinging – applies force to the gnome using that data

We use a Singleton pattern so any script can easily access input data.

Part 4: Creating a Reusable Singleton Class

A Singleton is a class that:

- Has only one instance in the scene

- Can be accessed from anywhere using .instance

This prevents duplicate managers and simplifies communication.

Step 1: Create the Singleton Script

1. Open the Scripts folder
2. Right-click → Create → C# Script
3. Name the script Singleton

Step 2: Add the Singleton Code

The Singleton script:

- Automatically keeps track of the active instance
- Allows other scripts to access it using ClassName.instance

You do not attach this script to a GameObject directly. It will be inherited by other manager scripts.

Part 5: Creating the InputManager Singleton

Step 1: Create the InputManager Object

1. Go to GameObject → Create Empty
2. Rename it InputManager

Step 2: Create and Attach InputManager Script

1. Select the InputManager object
2. Click Add Component
3. Type InputManager
4. Choose New Script
5. Confirm:
    - Name: InputManager
    - Language: C Sharp
6. Click Create and Add

Step 3: What InputManager Does

Every frame, InputManager:

- Reads data from the accelerometer
- Stores the left–right tilt value (X axis)
- Exposes it using a read-only property called sidewaysMotion

Read-only means:

- Other scripts can read the value
- They cannot change it accidentally Any script can now

access tilt data using: InputManager.instance.sidewaysMotion

Part 6: Creating the Swinging Script

This script applies sideways force to the gnome based on tilt input.

Step 1: Select the Gnome's Body

1. In the Hierarchy, expand Prototype Gnome
2. Select Prototype Body

Step 2: Create the Swinging Script

1. With the body selected, click Add Component
2. Type Swinging
3. Choose New Script
4. Name it Swinging.cs
5. Click Create and Add

Step 3: What the Swinging Script Does

- Runs during the physics update
- Checks if the object still has a Rigidbody2D
- Reads sidewaysMotion from InputManager
- Converts it into a Vector2 force

- Applies that force to the gnome's body

This makes the gnome swing naturally when the phone tilts.

Part 7: Testing the Tilt Controls

Step 1: Run the Game

1. Connect your phone
2. Open Unity Remote
3. Press Play ▶ in Unity

Step 2: Test Movement

- Tilt the phone left → gnome swings left
- Tilt the phone right → gnome swings right

If the gnome moves, the input system is working correctly

Input system complete!

You now have tilt-based controls working through Unity Remote.

Controlling the Rope

In this section, we will add on-screen buttons that allow the player to:

- Extend the rope (Down button)
- Retract the rope (Up button)

These buttons will work while the player is holding them down, not just tapping once. This is important because the rope needs to continuously change length while the finger is touching the screen.

How the Rope Buttons Work (Simple Explanation)

- We will use Unity UI Buttons for visuals
- We will NOT use the normal Button behavior
- Instead, we will use an Event Trigger component Why?
- Normal buttons only trigger when the finger is released
- We need:
    - One event when the finger touches down
    - Another event when the finger lifts up This allows the

rope to:

- Start extending/retracting when pressed
- Stop when released

Part 1: Creating the “Down” Button

Step 1: Add a UI Button

1. Go to GameObject → UI → Button
2. Unity will automatically create:
    - A Canvas
    - An EventSystem
3. Rename the button object to Down

You don’t need to modify the Canvas or EventSystem.

Step 2: Position the Button (Bottom-Right Corner)

1. Select the Down button
2. In the Inspector, find the Anchor Presets box (small square icon)
3. Hold:
    - Shift + Alt (Windows)
    - Shift + Option (Mac)
4. Click the Bottom-Right anchor

The button will snap to the bottom-right of the screen and stay there on all screen sizes.

Step 3: Change the Button Text

1. Expand the Down button in the Hierarchy
2. Select the child object named Text
3. In the Inspector, locate the Text component
4. Change the Text value to:
    - Down

The label on the button will update immediately.

Step 4: Remove the Default Button Component

1. Select the Down button (parent object)
2. In the Inspector, find the Button component
3. Click the ⚙ icon → Remove Component

Why remove it?

- Normal buttons only send events when released
- We need events for press and release

Part 2: Using Event Triggers Instead of Buttons

Step 5: Add an Event Trigger Component

1. With the Down button selected
2. Click Add Component

3. Search for Event Trigger

4. Add it

This component lets us respond to touch events.

Step 6: Add a Pointer Down Event (Start Extending Rope)

1. In the Event Trigger component, click Add New Event Type

2. Choose Pointer Down

3. Click the + button under the event Now configure

the event:

1. Drag the Rope object from the Hierarchy into the Object field

2. Open the Function dropdown

3. Choose:

    - Rope → isIncreasing

4. Check the checkbox ( )

This sets isIncreasing = true when the button is pressed.

Step 7: Add a Pointer Up Event (Stop Extending Rope)

1. Click Add New Event Type again

2. Choose Pointer Up

3. Click the + button

4. Drag the Rope object into the Object field
5. Choose:
    - Rope → isIncreasing
6. Uncheck the checkbox ( )

This sets isIncreasing = false when the finger lifts.

Step 8: Test the Down Button

1. Click Play ▶
2. Click and hold the Down button
    - Rope should extend
3. Release the button
    - Rope should stop extending If it

doesn't work:

- Check Pointer Down → isIncreasing = true
- Check Pointer Up → isIncreasing = false

Part 3: Creating the "Up" Button (Retract Rope)

The Up button works exactly the same way, but affects a different property.

Step 9: Add the Up Button

1. Go to GameObject → UI → Button
2. Rename it Up
3. Anchor it to Bottom-Right (same method as before)
4. Move it slightly above the Down button

Step 10: Change the Label to "Up"

1. Select the Text child object
2. Change the text to:
    - Up

Step 11: Remove Button Component and Add Event Trigger

1. Remove the Button component
2. Add an Event Trigger component

Step 12: Configure Rope Retraction Events

Add two events:

Pointer Down

- Function: Rope → isDecreasing
- Checkbox: Checked (true) Pointer

Up

- Function: Rope → isDecreasing

- Checkbox: Unchecked (false)

This makes the rope retract while the button is held.

Part 4: Testing

1. Press Play
2. Hold Down → rope extends
3. Hold Up → rope retracts
4. Tilt the phone (Unity Remote) to swing the gnome

You can now swing, extend, and retract the rope at the same time!

Making the Camera Follow the Gnome

Right now, when the rope becomes longer, the gnome can move so far down that it disappears from the screen. To fix this, we will make the camera move up and down automatically so it always follows the gnome.

We will do this by attaching a script to the Camera that copies the gnome's vertical (Y) position.

Part 1: Creating the Camera Follow Script

Step 1: Add the CameraFollow Script

1. In the Hierarchy, click on Main Camera.
2. In the Inspector, click Add Component.
3. Choose New Script.
4. Name the script CameraFollow and make sure the language is C#.

This script will now be attached to the camera.

How the CameraFollow Script Works

- The script runs inside LateUpdate.
- LateUpdate runs after all other objects have moved.
- This ensures the camera follows the gnome smoothly *after* the gnome's position has been updated.

The script will:

- Match the Y position of the camera to the gnome
- Prevent the camera from moving too far up or too far down
- Smooth the movement so the camera does not jump suddenly

To create smooth motion, the script uses Mathf.Lerp:

- A value close to 1 = faster camera movement
- A value close to 0 = slower, smoother movement

Step 2: Configure the CameraFollow Component

1. Select Main Camera again.
2. In the CameraFollow component:
    - Drag the Gnome → Body object into the Target field
3. Leave the other values (follow speed, limits) as default for now.

This tells the camera *which object to follow*.

Step 3: Test the Camera

1. Click Play.
2. Hold the Down button to lower the rope.
3. The camera should now follow the gnome downward.

When the rope is fully retracted, the camera will stop moving upward to avoid showing empty space.

Part 2: Visualizing Camera Limits

The script uses OnDrawGizmosSelected:

- This draws a line in the Scene view
- The line shows the top and bottom camera limits
- The line only appears when the camera is selected

If you adjust the topLimit or bottomLimit values in the Inspector, the line will change size.

This helps you visually understand where the camera can move.

Part 3: Introduction to Debugging Scripts

As your game grows, bugs are normal. Debugging helps you see what your code is doing while the game runs.

Unity scripts are best edited and debugged using:

- MonoDevelop or
- Visual Studio

In this guide, we use MonoDevelop.

Step 1: Open the Rope Script

1. Open Rope.cs in MonoDevelop.
2. Find the Update() method.
3. Locate this line:
4. if (topSegmentJoint.distance >= maxRopeSegmentLength)

Step 2: Add a Breakpoint

1. Click the gray margin to the left of the line.
2. A red dot will appear.

This means the debugger will pause the game when this line runs.

Step 3: Attach MonoDevelop to Unity

1. In MonoDevelop, click the Play button (top-left).
2. In the window that appears, click Attach.

MonoDevelop is now connected to Unity.

Step 4: Run the Game

1. Go back to Unity.
2. Click Play.
3. Hold the Down button.

Unity will freeze and MonoDevelop will open automatically.

ı This is normal — Unity is paused by the debugger, not crashed.

Inspecting Variables

At the bottom of MonoDevelop you will see:

- Locals Pane – shows variables currently in use
- Immediate Pane – lets you type C# commands Inspecting a

Variable

1. Expand topSegmentJoint in the Locals pane.
2. You can see its internal values, including distance.

In the Immediate pane, you can type: topSegmentJoint.distance

This shows the current rope segment distance.

Summary

- The camera now follows the gnome smoothly
- Camera movement is limited to avoid empty space
- You learned how to set breakpoints
- You learned how to inspect live variables

Congratulations!

Setting Up the Gnome's Code

At this stage, we finally prepare the Gnome to properly function in the game. The gnome needs to understand its own condition (alive, dead, damaged, holding treasure, etc.), while the Game Manager will handle the overall flow of the game.

Think of it like this:

- Gnome = manages *itself*
- Game Manager = manages *the entire game*

Part 1: BodyPart Script

Each body part (head, arms, legs) gets its own BodyPart script.

Purpose of BodyPart

- Change sprites based on damage type
- Store where blood effects should appear
- Disable physics once a detached limb stops moving Steps

1. Create a new C# script named BodyPart.cs
2. This script:
    - Requires a SpriteRenderer

- Responds to two damage types: Burning and Slicing
- Stores a reference to a Blood Fountain Origin (Transform)
- Detects when its Rigidbody2D falls asleep and removes physics

1 Note: This script references Gnome.DamageType, which will be created next. Errors at this point are expected.

Part 2: Gnome Script

The Gnome script manages the overall state of the character.

Steps

1. Create a new C# script named Gnome.cs
2. Add the Gnome script to the main Gnome object What the Gnome

Script Handles

- Tracks whether the gnome is alive or dead
- Tracks whether the gnome is holding treasure
- Changes arm sprites when treasure is picked up
- Spawns damage effects (blood or smoke)
- Detaches body parts on death
- Spawns a ghost after dying

- Schedules the gnome for removal

- You will see compiler errors related to

RemoveAfterDelay. This is normal and will be fixed shortly.

Part 3: Blood Fountain Setup

Blood effects need a position and direction for each detachable limb.

Steps

1. Create an empty GameObject named Blood Fountains
2. Make it a child of the main Gnome object
3. Inside it, create five empty child objects:
    - Head
    - Leg Rope
    - Leg Dangle
    - Arm Holding
    - Arm Loose
4. Position each object where blood should originate
5. Rotate each so the blue (Z) axis points opposite the spray direction

Connecting Blood Origins

- Select each body part

- Drag its matching Blood Fountain object into the Blood Fountain Origin field
- Do **not** assign one to the main Body (it does not detach)

Part 4: RemoveAfterDelay Script

This utility script removes objects after a short delay. Steps

1. Create a new C# script named RemoveAfterDelay.cs
2. This script:
    - Starts a coroutine when created
    - Waits for a set time
    - Destroys the object

This resolves earlier compiler errors in Gnome.cs.

Part 5: Configuring the Gnome Component

Select the Gnome prefab and configure:

- Camera Follow Target → Gnome Body
- Rope Body → Leg Rope
- Arm Holding Empty Sprite → Arm without treasure

- Arm Holding Treasure Sprite → Arm with gold
- Holding Arm Object → Arm Holding body part These values

will be used by the Game Manager.

Part 6: Resettable Script

Some objects must reset when the game restarts.

Steps

1. Create Resettable.cs
2. Add a UnityEvent inside the script
3. Add a public Reset() method that invokes the event

This allows objects to reset without writing custom reset code.

Part 7: Game Manager

The Game Manager controls the entire game lifecycle. Responsibilities

- Spawning gnomes
- Resetting the level
- Connecting rope and camera

- Handling traps, treasure, and exits
- Managing menus and pause state Steps

1. Create an empty GameObject named Game Manager

2. Add a new script called GameManager.cs Key Game

Manager Behaviors

- Start() immediately calls Reset()
- Reset():
    - Resets all Resettable objects
    - Creates a new gnome
    - Connects rope and camera
    - Unpauses the game
- KillGnome():
    - Shows damage effects
    - Detaches limbs
    - Disables player control
    - Resets the game after a delay
- Touch Handling:
    - Traps → kill gnome
    - Treasure → set holdingTreasure = true
    - Exit → win if holding treasure

Part 8: Preparing the Scene

Start Point

1. Create an empty GameObject named Start Point
2. Position it near the rope
3. Change its icon to a yellow capsule Create Gnome

Prefab

1. Drag the Gnome into the Gnome folder

2. Confirm prefab creation

3. Delete the gnome from the scene Configure

Game Manager

Assign:

- Starting Point → Start Point
- Rope → Rope object
- Camera Follow → Main Camera
- Gnome Prefab → Gnome prefab

Final Test

▶ Press Play

- A new gnome spawns at the start point
- Rope attaches correctly
- Camera follows the gnome
- Rope controls still work


You are now ready to add real gameplay elements like

traps and treasure!

Building Gameplay with Traps and Objectives

Now that the core systems are working (gnome, rope, camera, and game manager), we can finally add real gameplay elements:

- Traps that kill the gnome
- Treasure that must be collected
- An exit that ends the game

From this point onward, most development becomes level design rather than heavy coding.

Core Idea: Detecting Touches

Many gameplay elements need to react when the player (the gnome) touches them. To avoid writing the same code repeatedly, we'll use one reusable script that detects collisions and triggers events.

This script will:

- Detect when something tagged Player touches it
- Fire a Unity Event
- Let us decide what happens *in the Inspector*, not in code

Part 1: SignalOnTouch Script

Purpose

- Detect when the gnome touches something
- Notify other systems using Unity Events Steps

1. Create a new C# script named SignalOnTouch.cs
2. This script:
    - Listens for OnCollisionEnter2D and OnTriggerEnter2D
    - Checks if the touching object has the Player tag
    - Invokes a Unity Event if the tag matches This script will be

reused for:

- Traps
- Treasure
- Exit

Part 2: Creating a Simple Trap (Spikes)

Steps

1. Import the level object sprites from Sprites/Objects
2. Drag SpikesBrown into the scene
3. Configure the spikes:
    - Add PolygonCollider2D
    - Add SignalOnTouch

4. Configure the SignalOnTouch event:

    - Drag Game Manager into the event slot
    - Choose GameManager.TrapTouched

5. Turn the spikes into a prefab:

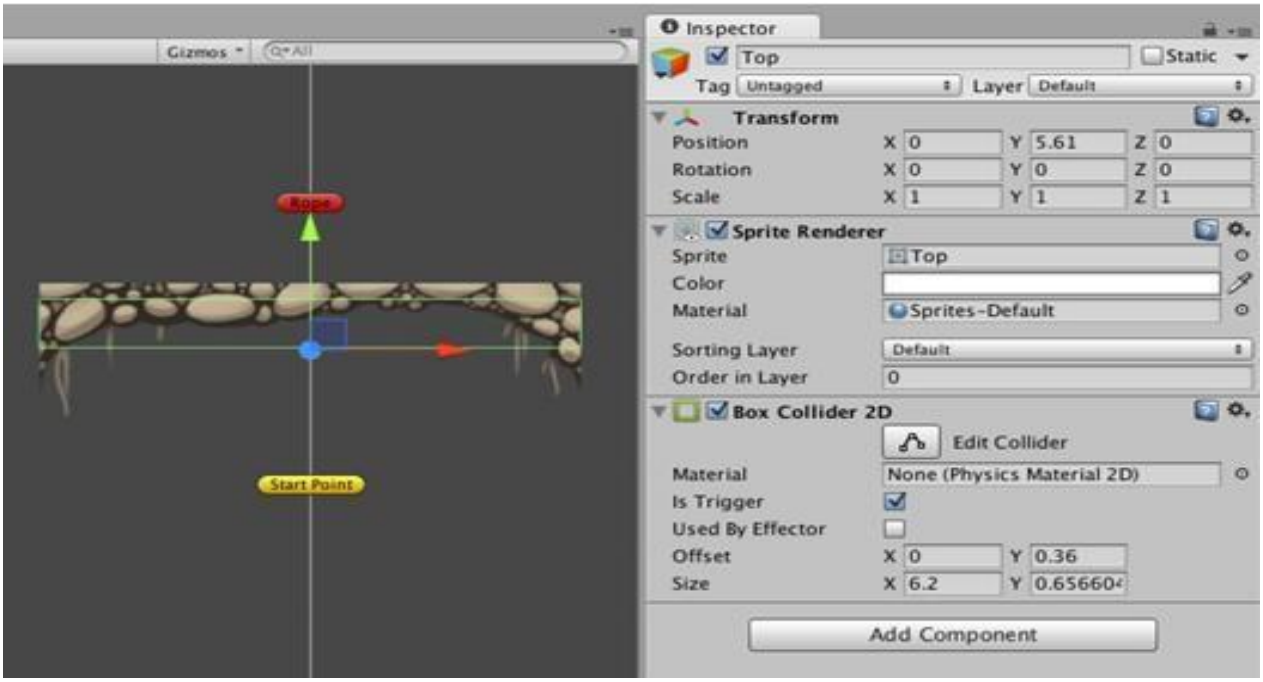
    - Drag the spikes from the Hierarchy into the Level folder

Test

- Run the game

    - Touch the spikes
    - The gnome dies and respawns

Trap system success!



Part 3: Creating the Exit

The exit is placed at the top of the level and ends the game *only if the gnome is holding treasure*.

Steps

1. Import the Sprites/Background folder
2. Drag the Top sprite into the scene
3. Position it just below the Rope Configure the

Exit

- Add BoxCollider2D
- Enable Is Trigger
- Resize the collider so it is wide and short Add Touch Detection
- Add SignalOnTouch
- Configure the event:
    - Object: Game Manager
    - Function: GameManager.ExitReached

Part 4: SpriteSwapper Script

We need a reusable way to **swap sprites** (e.g., treasure present → treasure gone).

Steps

1. Create a new C# script named SpriteSwapper.cs What It Does

- Stores the original sprite
- Changes to a new sprite when SwapSprite() is called
- Restores the original sprite when ResetSprite() is called

This script will be used for treasure and later polish effects.

Part 5: Creating the Treasure

Steps

1. Drag TreasurePresent into the scene
2. Place it near the bottom of the well Configure the

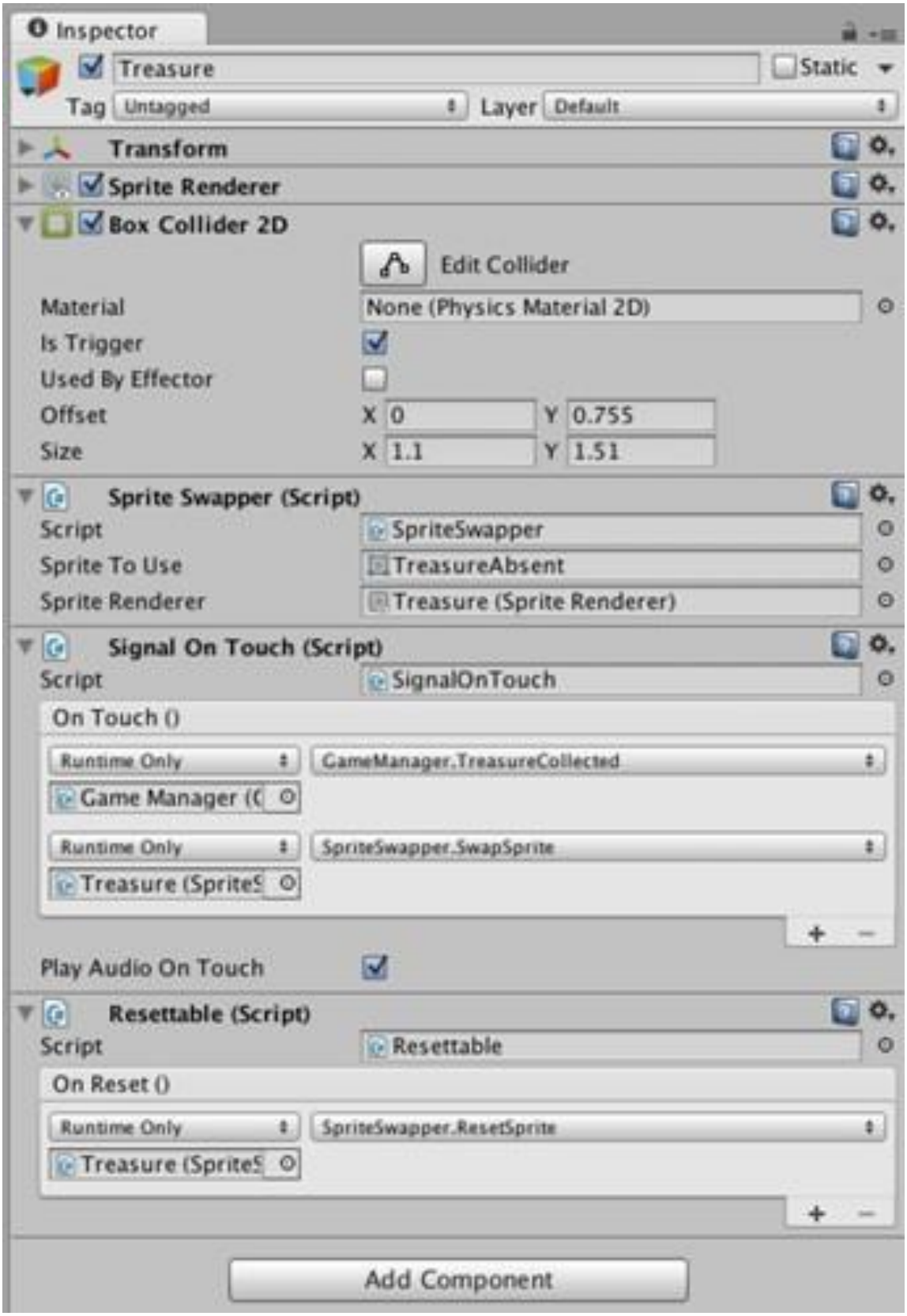
Treasure

- Add BoxCollider2D
- Enable Is Trigger Add

Sprite Swapping

- Add SpriteSwapper
- Assign:
    - Sprite Renderer → TreasurePresent
    - Sprite To Use → TreasureAbsent Add

Touch Detection


Inspector
Treasure
Static
Tag
Untagged
Layer
Default
Transform
Sprite Renderer
Box Collider 2D
Edit Collider
Material
None (Physics Material 2D)
Is Trigger
Used By Effector
Offset
X 0
Y 0.755
Size
X 1.1
Y 1.51
Sprite Swapper (Script)
Script
SpriteSwapper
Sprite To Use
TreasureAbsent
Sprite Renderer
Treasure (Sprite Renderer)
Signal On Touch (Script)
Script
SignalOnTouch
On Touch ()
Runtime Only
GameManager.TreasureCollected
Game Manager (
Runtime Only
SpriteSwapper.SwapSprite
Treasure (Sprite
Play Audio On Touch
Resettable (Script)
Script
Resettable
On Reset ()
Runtime Only
SpriteSwapper.ResetSprite
Treasure (Sprite
Add Component

- Add SignalOnTouch
- Add two events:
    1. GameManager.TreasureCollected
    2. SpriteSwapper.SwapSprite Add

Reset Support

- Add Resettable
- Add one event:
    - SpriteSwapper.ResetSprite

This ensures the treasure resets when the game restarts.

Final Test

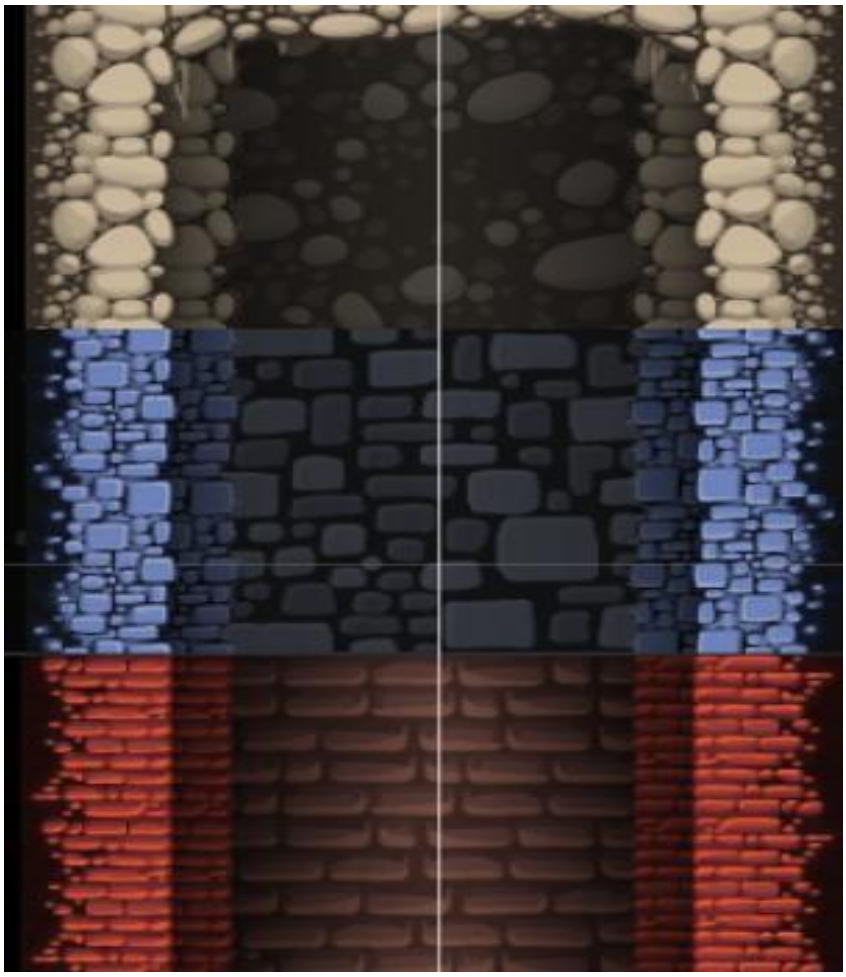
- Run the game
    - Touch spikes → gnome dies
    - Touch treasure → treasure disappears and arm changes
    - Die → treasure reappears
    - Reach exit with treasure → win condition triggered

You now have a complete gameplay loop!

Next up: polishing the experience with visuals, audio, and more traps.

Adding a Background

At this point, the gnome and level objects are visible, but they are still placed against Unity's default blue background. To make the game easier on the eyes (and closer to a finished product), we will add a simple temporary background. This will later be replaced with a proper background sprite during the art-polishing stage.



Part 1: Adding the Background

Step 1: Add the background quad

- In the Unity menu bar, click GameObject → 3D Object → Quad.
- Rename the new object to Background.

Step 2: Move the background behind the level

- Select the Background object in the Hierarchy.
- In the Transform component, set the Z position to 10.

Even though this is a 2D game, Unity still uses a 3D space. By moving the background further back on the Z-axis, we ensure it appears *behind* all sprites instead of covering them.

Step 3: Resize and position the background

- Press T on your keyboard to switch to the Rect Tool.
- Use the resize handles to stretch the quad so that:
  - The top edge lines up with the top of the level sprite.
  - The bottom edge lines up with the treasure near the bottom of the well.

Test the game

- Press Play.

- You should now see a simple gray background behind the entire level instead of the default blue.

Wrapping Up

At this stage, the core gameplay of the game is complete. A lot of important systems are now working together:

- The gnome is physically simulated and attached to a simulated rope.
- The rope can be raised and lowered using on-screen buttons.
- The camera follows the gnome so it always stays visible.
- The gnome responds to device tilt and moves left and right.
- Traps can kill the gnome and trigger a respawn.
- Treasure can be collected and visually changes state.
- An exit allows the player to win once the treasure is collected.

Functionally, the game is now playable from start to finish. However, it is still very rough visually: the gnome is a stick figure, the environment is simple, and the background is temporary.

Next, the focus will shift from gameplay systems to visual polish, improving sprites, animations, and overall presentation to make the game look and feel much better.

Polishing the Game

By the end of this chapter, you will have refined Gnome's Well That Ends Well so that it looks, feels, and sounds like a finished game rather than a prototype.

Areas of Polish

This chapter focuses on three main areas: Visual Polish

- Replacing the stick-figure gnome with hand-painted sprites
- Improving the background and environment
- Adding visual depth using layers and foreground elements

Gameplay Polish

- Improving traps and overall game flow
- Adding tools that help with gameplay testing (such as invincibility)

Audio Polish

- Adding sound effects that react to player actions

All resources used in this chapter can be found in the asset package available from the book's website.

Part 1: Updating the Gnome's Art

The first step in polishing the game is replacing the prototype gnome sprites with proper artwork.

Step 1: Importing the New Sprites

1. Copy the GnomeParts folder from the provided resources into your project's Sprites folder.
2. Inside this folder are two subfolders:
    - Alive – sprites used while the gnome is alive
    - Dead – sprites used when the gnome dies

Note: The asset pack contains extra sprites that are not required for this tutorial. These are optional and can be used for further improvements.

Step 2: Preparing the Sprites

Before using the new sprites, they must be configured correctly.

Converting to Sprites

1. Select all sprites in the Alive folder.
2. In the Inspector, ensure Texture Type is set to Sprite (2D and UI).

Setting Pivot Points

For every sprite *except the Body*:

1. Select the sprite.
2. Click Sprite Editor.
3. Move the pivot point (blue circle) to the joint where the body part should rotate.

Correct pivot placement ensures natural rotation during physics simulation.

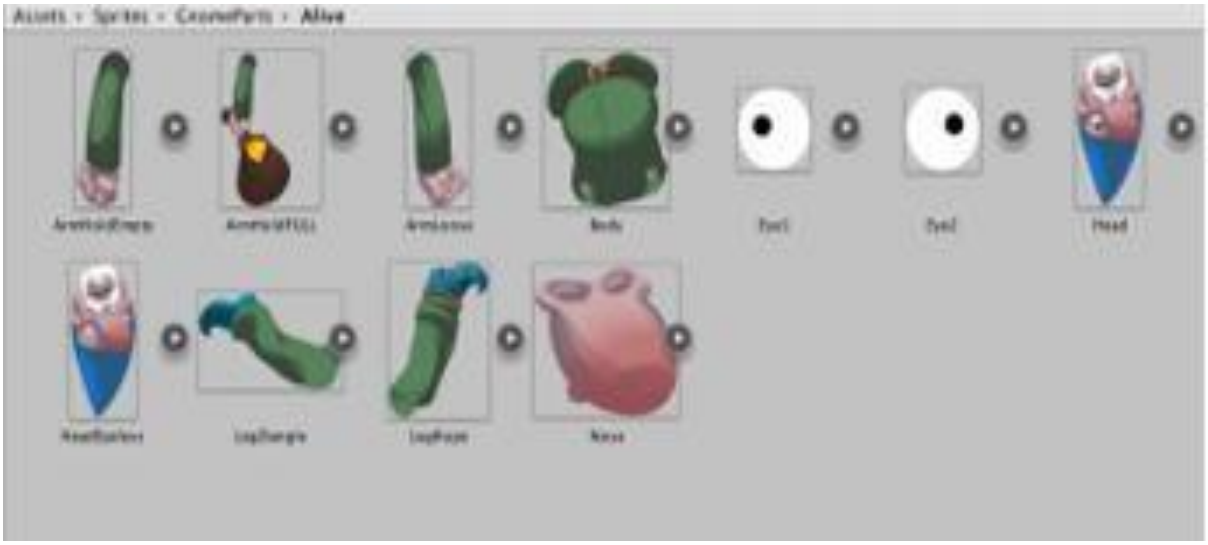
Step 3: Creating the Updated Gnome Prefab

To keep the old prototype intact, we create a new prefab.

1. Duplicate the prototype gnome prefab and rename it Gnome.
2. Drag the new prefab into the Scene.
3. Replace each body part's sprite with the corresponding sprite from the Alive folder.

At this stage, the gnome's parts may look misaligned. This will be fixed next.

Part 2: Positioning and Sorting the Body Parts



Step 1: Repositioning

1. Move the Head so the neck aligns with the shoulders.
2. Adjust Arms to match the shoulder pivots.
3. Adjust Legs to align with the waist.

Purple dots on the Body sprite indicate correct joint locations.

Step 2: Sprite Sorting Order

To ensure correct visual layering:

- Set Head and Arms → Order in Layer = 2
- Set Body → Order in Layer = 1

This ensures the head appears on top, followed by arms, then body.

Part 3: Updating the Physics

The new sprites require updated colliders and joint settings.

Step 1: Replacing Colliders

1. Remove existing colliders:
    - Remove Box Collider 2D from arms and legs
    - Remove Circle Collider 2D from the head
2. For each arm, leg, and head:

- Create an empty child object named Collider (Position: 0,0,0)
- Add a Polygon Collider 2D
- Click Edit Collider and manually adjust the shape to fit the sprite

Polygon colliders should roughly match the sprite without overlapping other parts.

Step 2: Body Collider Adjustment

- Increase the Body's Circle Collider radius to 1.2 to match its larger shape.

Step 3: Updating Joints

Each body part is connected using hinge joints.

1. For each body part (except the body):
    - Move the Anchor and Connected Anchor to the correct pivot point
    - Arms pivot at shoulders
    - Legs pivot at hips
    - Head pivots at the neck

Tip: Dragging near the sprite's center causes anchors to snap into place.

Don't forget the Leg Rope, which has two joints. Move the rope joint anchor to the ankle.

Part 4: Updating the Gnome Script

The Gnome script still references old arm sprites.

1. Select the parent **Gnome** object.
2. Assign:
    - ArmHoldEmpty → Arm Holding Empty
    - ArmHoldFull → Arm Holding Full

This ensures the correct arm sprite is shown when carrying or dropping treasure.

Part 5: Finalizing the Gnome

1. Scale the Gnome:
    - Change X and Y scale from 0.5 to 0.3
2. Click Apply to save changes to the prefab.
3. Remove the gnome instance from the Scene.

Step 1: Updating the Game Manager

1. Select the Game Manager.

2. Assign the updated Gnome prefab to the Gnome Prefab slot.

3. Play the game to confirm the new gnome appears correctly.

Next, the focus will shift from gameplay systems to visual polish, improving sprites, animations, and overall presentation to make the game look and feel much better.

Unity Game UI Tutorial

In this tutorial, we will improve the look of your game's UI by updating buttons, adding a Game Over screen, Pause menu, and a developer "Invincibility" mode toggle. We'll make every step easy to follow with extra details.

Part 1: Prepare Your Sprites

1. Open Unity and go to your Project window.
2. Make sure you have a Sprites folder. Inside it, create a folder called Interface.
3. Import all the UI sprites (images) for this section into the Interface folder. Do NOT include the "You-Win" sprite when doing the next step.
4. Select all imported images (except "You-Win") → in the Inspector change Pixels Per Unit to 2500.
    - This ensures the images will display at the correct size in the UI.

Step 1: Update Up and Down Buttons

Your game already has default buttons at the bottom-right. Let's make them look nicer.

1. Remove the old text label

- In the Hierarchy, find Down Button → expand it.

- Delete the Text child object.

2. Change the sprite

- Select the Down Button → in the Inspector, find Image (Script) → Source Image → select the Down sprite from your Interface folder.
- Click Set Native Size. The button will automatically resize.
- Move it back to the bottom-right corner.

3. Repeat for Up Button

- Delete the Text child.
- Set Source Image to Up sprite → click Set Native Size.
- Move the Up button just above the Down button.

4. Test the buttons

- Press Play → check if both buttons still work and look nicer.

Step 2: Group Buttons into a Container

This keeps your UI organized and allows enabling/disabling all buttons together.

1. Create container
    - Right-click Canvas → Create Empty → rename it Gameplay Menu.
2. Fill the screen

- Select Gameplay Menu → in Rect Transform click Anchors → choose stretch horizontally and vertically.
- Set Left, Right, Top, Bottom to 0.

3. Move buttons into container

- Drag Up Button and Down Button into Gameplay Menu in the Hierarchy.

Step 3: Create Game Over Screen

1. Create container

- Right-click Canvas → Create Empty → rename it Game Over.
- Stretch it to fill the screen like Gameplay Menu.

2. Add "You Win" Image

- Right-click Game Over → UI → Image.
- Stretch image to fill the container.
- Set Left/Right Margins = 30, Bottom Margin = 60.
- Source Image → select You-Win sprite → enable Preserve Aspect.

3. Add New Game Button

- Right-click Game Over → UI → Button.
- Rename → New Game.
    - Set text → "New Game".

- Set anchors → Bottom-Center. Move it to bottom- center of screen.

4. Connect button to GameManager

   - Select New Game Button → in Button (Script) → On Click () → click + → drag Game Manager into slot → select GameManager → RestartGame function.

5. Test

   - Play the game → reach the exit → Game Over screen appears → click "New Game" resets game.

Part 1: Pause Menu

Step 1: Add Pause Button

- Right-click Gameplay Menu → UI → Button → rename Menu Button.
- Remove Text child.
- Set Source Image → Menu sprite → click Set Native Size.
- Set anchors → Top-Right → move button to top-right corner.

1. Connect Pause Button to GameManager

   - On Click () → + → drag Game Manager → select GameManager → SetPaused → check true.

2. Create Main Menu (Pause Menu)

- Right-click Canvas → Create Empty → rename Main Menu → stretch to fill screen.
- Add two buttons as children:
    - Restart Button → text: “Restart Game”
    - Resume Button → text: “Resume Game”

3. Connect buttons

- Restart Button → GameManager.RestartGame
- Resume Button → GameManager.Reset

4. Connect Main Menu to GameManager

- Drag Main Menu object into GameManager’s Main Menu slot.

5. Test

- Play → tap Pause → Menu appears → Resume works → Restart works.

Step 2: Invincibility Mode Toggle

This is a developer tool that makes your gnome invincible for testing.

1. Create Debug Menu
    - Right-click Canvas → Create Empty → rename Debug Menu → stretch to fill screen.
2. Add Toggle

- Right-click Debug Menu → UI → Toggle → rename Invincible → anchors: Top-Left → move to top-left corner.

3. Configure Toggle
    - Select Label child → text: “Invincible” → color: white.
    - Set Is On → off.
4. Connect to GameManager
    - On Value Changed () → + → drag Game Manager → select GameManager.gnomeInvincible.
5. Test
    - Play → toggle Invincible → gnome won’t die when touching traps.

Wrapping Up

- Up/Down buttons look nicer and work.
- Game Over screen shows when player wins.
- Pause menu works with Restart/Resume buttons.
- Developer toggle allows invincibility for easy testing.

The game UI is now polished and beginner-friendly!

Final Touches Tutorial

In this tutorial, we'll add more variety to the level by creating new traps, blocks, particle effects, a main menu, and audio, so the game feels polished and complete.

Part 1: Prepare Your Sprites & Prefabs

Before adding new traps and blocks:

1. Make sure all sprites for spikes, spinning blades, blocks, and particles are imported into Unity.
2. Organize them in folders inside your Sprites folder for easy access:
    - Spikes → brown, blue, red
    - Spinner → arm, hub, blades
    - Blocks → square and long, blue/red/brown
    - Particles → blood textures

Step 1: Add More Spikes

We're adding blue and red spikes as variations of the brown spikes.

1. Duplicate the existing spike prefab
    - Select SpikesBrown prefab → press Ctrl+D (Cmd+D on Mac) → rename it SpikesBlue.

- Duplicate again → rename it SpikesRed.

2. Change the sprite
    - Select SpikesBlue prefab → in Sprite Renderer → Sprite, choose SpikesBlue image.
3. Update the collider
    - Select the Polygon Collider 2D → click Gear icon → Reset.
    - This makes the collider match the new sprite shape.
4. Repeat for SpikesRed
    - Select SpikesRed prefab → change sprite → reset collider.
5. Place new spikes in the level
    - Drag SpikesBlue/Red into the scene wherever you want more variety.

Step 2: Add Spinning Blade Trap

The spinning blade is a moving trap that kills the gnome.

2.1 Build the Spinner

1. Drag SpinnerArm sprite into the scene → set Sorting Layer = Level Objects.
2. Add SpinnerBladesClean sprite → child of SpinnerArm → set Order in Layer = 1, position at the top center.

3. Add SpinnerHubcab sprite → child of SpinnerArm → Order in Layer = 2, X = 0.

2.2 Add Damage

1. Select SpinnerBladesClean → add Circle Collider 2D → radius = 2.
2. Add SignalOnTouch script → OnTouch event: drag GameManager → function = GameManager.TrapTouched.

2.3 Make Blades Spin

1. Add Animator component to SpinnerBladesClean.
2. In Level folder, create:
    - Animator Controller → Spinner
    - Animation → Spinning
3. Assign Animator Controller → SpinnerBladesClean → Controller = Spinner.
4. Open Animation window → select Spinning → add property Transform → Rotation → Z.
    - Left keyframe = 0°
    - Right keyframe = 360°
    - Adjust timeline for speed → make it loop → select Loop Time.
5. Scale spinner to fit the level → SpinnerArm scale = 0.4, 0.4.
6. Make a prefab → drag SpinnerArm into Project folder → rename Spinner.

Step 3: Add Blocks

Blocks don't kill the gnome but act as obstacles.

1. Drag BlockSquareBlue, BlockSquareRed, BlockSquareBrown, BlockLongBlue, BlockLongRed, BlockLongBrown into the scene.
2. Add Box Collider 2D to each block.
3. Convert each block to a prefab → drag into Level folder.
4. Place blocks in the level to create paths or barriers.

Part 2: Add Particle Effects

We're adding blood effects for when the gnome dies.

Step 1 Prepare Blood Material

1. Select Blood texture → change type = Default, enable Alpha Is Transparency.
2. Create new Material → name Blood → Shader = Unlit → Transparent → assign Blood texture.

Step 2 Blood Fountain (continuous)

1. GameObject → Effects → Particle System → rename Blood Fountain.
2. Set Particle System parameters:
    - Color over lifetime: alpha 100 → 0


Particle System
Open Editor...
Blood Fountain
Duration 0.10
Looping
Prewarm
Start Delay 0
Start Lifetime 5
Start Speed 10
3D Start Size
Start Size 0.05 0.3
3D Start Rotation
Start Rotation 0 0
Randomize Rotation 0
Start Color
Gravity Modifier 2
Simulation Space World
Simulation Speed 1
Delta Time Scaled
Scaling Mode Shape
Play On Awake*
Max Particles 1000
Auto Random Seed
Emission
Rate over Time 50
Rate over Distance 0
Bursts
Time Min Max Cycles Interval
List is Empty
Shape
Shape Cone
Angle 3.5
Radius 0.05
Radius Thickness 1
Arc 360
Mode Random
Spread 0
Length 5
Emit from Base
Position X 0 Y 0 Z 0
Rotation X 0 Y 0 Z 0
Scale X 1 Y 1 Z 1
Align To Direction
Randomize Direction 0
Spherize Direction 0
Randomize Position 0

3. Make prefab → drag into Gnome folder.

Step 3 Blood Explosion (single burst)

1. Create another Particle System → Blood Explosion.
2. Configure emission → circle emitter → emit all particles at once.
3. Add RemoveAfterDelay script → Delay = 2 sec.
4. Make prefab → drag into Gnome folder.

Step 4 Connect Particles to Gnome

1. Select Gnome prefab → assign Blood Explosion → Death Prefab slot.
2. Assign Blood Fountain → Blood Fountain slot.
3. Test → gnome touches trap → blood appears.

Step 4: Main Menu

1 Create Menu Scene

1. File → New Scene → save as Menu.
2. Add background: UI → Image → Source Image = Main Menu Background → stretch horizontally, preserve aspect.

2 Add New Game Button

1. UI → Button → rename New Game → anchors = bottom- center.
2. Position X = 0, Y = 40 → width = 160, height = 30.
3. Change label → "New Game".

Step 5: Loading Overlay

1. Create empty → Loading Overlay → child of Canvas → stretch full screen.
2. Add Image → color = black, alpha 50%.
3. Add Text child → centered → font size bigger → text = "Loading…" → white color.

Step 6: Connect Menu to Game

1. Add MainMenu script to Main Camera → set:
    - Scene to Load = Main
    - Loading Overlay = Loading Overlay object
2. Select New Game button → OnClick → MainMenu.LoadScene.
3. Build Settings → File → Build Settings → add Menu and Main scenes → Menu first.
4. Test → click New Game → loading overlay → game scene opens.

Step 7: Add Audio

1 Add Audio Sources

- Spikes → "Death by Static Object"
- Spinner → "Death by Moving Object"
- Treasure → "Treasure Collected"

- Game Manager → Gnome Died / You Win

2 Settings

- Loop = off
- Play On Awake = off

3 Test

- Play → gnome dies, collects treasure, wins → hear sounds.

Wrapping Up

- All traps, blocks, and particle effects added.
- Main Menu and Loading Overlay ready.
- Audio integrated.
- Game feels polished and complete.

Conclusion: Lessons Learned from Making the Game

Creating *Gnome's Well* demonstrates how a complete game is built by combining many small systems into a cohesive experience. Through this project, several key lessons emerge:

- Game development is iterative—features are added, tested, and refined repeatedly.
- Organization matters—using prefabs, folders, and menus keeps projects manageable.
- Visual and audio feedback greatly improves player experience.
- Developer tools, such as debug modes, make testing faster and more efficient.
- Even simple mechanics can result in engaging gameplay when combined thoughtfully.

Most importantly, this project shows that building a full game is achievable for beginners when broken into clear, manageable steps. The skills learned here form a strong foundation for creating more complex games in the future.